

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Athavan k	Reg. No.:	192524036
Section / Batch:	AI&DS	Date:	24/7/26

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

1) A payroll system uses Binary Search on sorted employee records, but frequent insertions disturb order. a) Explain Binary Search operation. b) Analyze the impact of unsorted data. c) Discuss maintaining sorted data for efficiency.

2) A system log analyzer scans large logs to detect patterns using brute-force string matching. a) Explain the working of the algorithm. b) Analyze its time complexity. c) Justify the need for optimized approaches

Code of Conduct

I Athavan k(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Athavan k

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46
Overall Total (100)						86

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1:

A payroll system uses Binary Search on sorted employee records, but frequent insertions disturb order.

a) How Binary Search Operates on the Payroll Records

The payroll system keeps employee records arranged by a key value, usually the employee ID, in strictly increasing order. Binary Search takes advantage of this order instead of checking every record one by one. It starts by looking at the record sitting exactly in the middle of the array. If that record's ID matches the one being searched for, the lookup ends immediately. If the target ID is smaller than the middle record's ID, the algorithm knows the employee, if present, must lie somewhere in the left half of the array, so it discards the entire right half in one step. If the target is larger, it discards the left half instead. This halving repeats on the remaining segment until either the record is found or the segment shrinks to nothing, which means the employee ID does not exist in the system.

The core idea can be written out as a short trace:

```
low = 0, high = n - 1
while low <= high:
    mid = (low + high) / 2
    if records[mid].id == targetID: return mid
    elif records[mid].id < targetID: low = mid + 1
    else: high = mid - 1
return NOT_FOUND
```

Because each comparison eliminates roughly half of the remaining candidates, even a payroll file with a hundred thousand employees can be searched in about seventeen comparisons, which is why HR software relies on this technique for instant lookups.

b) Impact of Unsorted or Disturbed Data on Performance

The entire efficiency of Binary Search depends on one assumption: the array stays sorted. The moment a new employee joins and their record is simply appended to the end of the list rather than inserted at the correct sorted position, that assumption breaks. Binary Search has no way to detect this on its own — it will still pick a midpoint and compare, but the decision to discard the left or right half is only valid when the array is ordered. On a disturbed array, that decision can throw away the very half that contains the record

being searched for, so the algorithm can return a wrong 'not found' result even though the employee's record is sitting in the array.

If the payroll team tries to work around this by re-sorting the file before every single search, the cost of the search stops being $O(\log n)$ and becomes dominated by the sort itself, which is typically $O(n \log n)$ using an efficient comparison-based sort. If they instead fall back to a plain sequential scan to stay safe on unsorted data, every lookup degrades to $O(n)$ in the worst case, since the target ID could be the very last record checked. In a payroll system that runs searches every time a slip is generated or a query is raised, this difference is significant: a system with ten thousand employees would need on the order of fourteen comparisons when sorted, but potentially all ten thousand when it is not.

Case	Sorted Array (Binary Search)	Unsorted / Disturbed Array
Best Case	$O(1)$	$O(1)$ only by chance
Average Case	$O(\log n)$	$O(n)$ — needs re-sort or linear fallback
Worst Case	$O(\log n)$	$O(n \log n)$ for re-sort + $O(\log n)$ search, or $O(n)$ if searched without re-sorting

c) Why Maintaining Sorted Data Matters for Efficiency

Keeping the payroll array sorted is not a cosmetic choice — it is the precondition that makes fast lookups possible in the first place. The practical fix is to never let a plain append happen. When a new employee is added, the system should locate the correct insertion position, which Binary Search itself can find in $O(\log n)$ time, and then shift the later elements over by one slot to make room, an $O(n)$ operation for an array-based structure. This keeps every subsequent search running at $O(\log n)$ instead of falling back to $O(n)$ or paying for repeated full re-sorts.

For a payroll system where insertions are frequent, it is often worth going a step further and storing the records in a self-balancing structure such as a balanced binary search tree or a B-tree, both of which keep data ordered automatically and support insertion in $O(\log$

n) time without the $O(n)$ shifting cost of an array. The underlying lesson is the same either way: the benefit of Binary Search's logarithmic speed only holds as long as the ordering invariant is actively protected, and any process that adds data has to preserve that order rather than assume someone will clean it up later.

d) Full C Program

The program below lets the user enter the sorted list of employee IDs, then repeatedly search for an ID or insert a new one — the insertion routine keeps the array sorted at all times, which is exactly the safeguard discussed in part (c).

```
#include <stdio.h>

int binarySearch(int arr[], int n, int target, int *comparisons) {
    int low = 0, high = n - 1;
    *comparisons = 0;
    while (low <= high) {
        int mid = (low + high) / 2;
        (*comparisons)++;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

void insertSorted(int arr[], int *n, int value) {
    int i = *n - 1;
    while (i >= 0 && arr[i] > value) {
        arr[i + 1] = arr[i];
        i--;
    }
    arr[i + 1] = value;
    (*n)++;
}

int main() {
```

```

int arr[100], n, choice, target, comparisons, result;

printf("Enter the number of employee IDs: ");
scanf("%d", &n);

printf("Enter %d employee IDs in sorted order:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

do {
    printf("\n--- Payroll Binary Search Menu ---\n");
    printf("1. Search for an employee ID\n");
    printf("2. Insert a new employee ID (keeps array sorted)\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter the employee ID to search: ");
            scanf("%d", &target);
            result = binarySearch(arr, n, target, &comparisons);
            if (result != -1)
                printf("Employee ID %d found at position %d (in %d comparisons).\n",
target, result + 1, comparisons);
            else
                printf("Employee ID %d not found (in %d comparisons).\n", target,
comparisons);
            break;
        case 2:
            printf("Enter the new employee ID to insert: ");
            scanf("%d", &target);
            insertSorted(arr, &n, target);
            printf("Inserted. Array remains sorted with %d records.\n", n);
            break;
        case 3:
            printf("Exiting program.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
}

```

```
    }  
    } while (choice != 3);  
  
    return 0;  
}
```

Sample run: with IDs 101, 102, 105, 108, 110 entered, searching for 108 returns 'found at position 4 (in 2 comparisons)'; inserting 106 keeps the list sorted as 101, 102, 105, 106, 108, 110, and a later search for 106 correctly returns 'found' instead of failing.

Question 2:

A system log analyzer scans large logs to detect patterns using brute-force string matching.

a) How the Brute-Force Pattern Matching Algorithm Works

When the log analyzer looks for a suspicious pattern, such as a specific error code or an intrusion signature, inside a large log file, the brute-force approach treats the log text and the pattern as two plain strings and compares them character by character without any pre-processing or shortcuts. It places the pattern against the very start of the log text and checks whether every character of the pattern matches the corresponding character of the text at that position. If all characters match, an occurrence has been found at that position. If any character mismatches, the algorithm does not try to be clever about it — it simply slides the pattern one position to the right in the log text and starts the character-by-character comparison all over again from the beginning of the pattern.

This process repeats for every possible starting position in the log file, right up until there are fewer characters left in the text than the length of the pattern, at which point no further match is possible. The logic can be summarized as:

```
for i in range(0, n - m + 1):  
    j = 0  
    while j < m and text[i + j] == pattern[j]:  
        j += 1  
    if j == m:  
        report match at position i
```

The method is called 'brute-force' precisely because it makes no attempt to reuse information from a previous failed comparison; every new alignment starts completely fresh, which is what makes it simple to implement but costly to run on large inputs.

b) Time Complexity Analysis

Let n represent the length of the log file and m represent the length of the pattern being searched for. In the best case, most mismatches happen at the very first character of an attempted alignment, so each of the roughly n positions only costs a single comparison, giving a best-case running time of $O(n)$. The worst case is far more expensive: it occurs when the text and pattern share long runs of repeated characters, so that many alignments

match almost the entire pattern before finally failing on the last character. In that situation, the algorithm performs close to m comparisons at each of the $n - m + 1$ possible starting positions, which pushes the running time to $O(n \times m)$.

Parameter	Value / Behaviour
Text length (n) / Pattern length (m)	n = size of the log file, m = size of the pattern being searched
Best Case	$O(n)$ — mismatch found in the first character of most attempts
Worst Case	$O(n \times m)$ — every alignment matches almost fully before failing
Space Complexity	$O(1)$ extra space, no pre-processing of the pattern

For a genuine system log analyzer this worst case is not just a theoretical curiosity. Log files are full of repetitive structure — timestamps, recurring field labels, and repeated substrings such as 'error', 'warning', or IP address fragments — which is exactly the kind of input that pushes brute-force matching toward its $O(n \times m)$ worst-case behaviour rather than its optimistic $O(n)$ best case.

c) Why Optimized Pattern-Matching Approaches Are Needed

A production log analyzer typically has to scan gigabytes of log data every day while searching for several patterns simultaneously, and it often needs to do this in near real time so that alerts fire before an intrusion or failure spreads. Brute-force matching's $O(n \times m)$ worst case makes it a poor fit for that workload: doubling the log size or the pattern length can noticeably slow down every scan, and the wasted, repeated comparisons on partially-matching text add up quickly across millions of log lines.

This is exactly why real systems move to algorithms that remember information from previous comparisons instead of restarting blindly. The Knuth-Morris-Pratt algorithm builds a failure function so that after a partial match it can skip ahead without re-checking characters it has already confirmed, giving a guaranteed $O(n + m)$ running time. The Boyer-Moore algorithm compares from the end of the pattern and uses bad-character and good-suffix rules to skip large sections of text that cannot possibly contain a match, which performs especially well on the kind of long, structured text found in logs. For

scanning against many patterns at once, such as a whole library of known attack signatures, the Aho-Corasick algorithm builds a single automaton so that all patterns are checked together in one linear pass. Adopting one of these approaches is what allows a log analyzer to stay responsive as log volume and the number of tracked patterns grow, rather than slowing down linearly, or worse, with every additional pattern it has to watch for.

d) Full C Program :

The program below takes the log text and the pattern directly from the user, runs the brute-force scan, and prints every index at which the pattern occurs along with the total number of character comparisons performed — a practical demonstration of the $O(n \times m)$ behaviour discussed in part (b).

```
#include <stdio.h>
#include <string.h>

void bruteForceSearch(char text[], char pattern[]) {
    int n = strlen(text);
    int m = strlen(pattern);
    int comparisons = 0;
    int matches = 0;

    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m) {
            comparisons++;
            if (text[i + j] != pattern[j])
                break;
            j++;
        }
        if (j == m) {
            printf("Pattern found at index %d\n", i);
            matches++;
        }
    }

    printf("\nTotal occurrences found: %d\n", matches);
    printf("Total character comparisons made: %d\n", comparisons);
}
```

```
}  
  
int main() {  
    char text[1000], pattern[100];  
  
    printf("Enter the log text: ");  
    fgets(text, sizeof(text), stdin);  
    text[strcspn(text, "\n")] = '\0';  
  
    printf("Enter the pattern to search for: ");  
    fgets(pattern, sizeof(pattern), stdin);  
    pattern[strcspn(pattern, "\n")] = '\0';  
  
    bruteForceSearch(text, pattern);  
  
    return 0;  
}
```